Frontend Interview Handbook

Middle Frontend Developer

100 вопросов и ответов · English + Русский

JavaScript · TypeScript · React · React Hooks · Redux Toolkit · Next.js

Mini-reference for interview preparation

Содержание / Contents

JavaScript — вопросы 1–28 (28)

TypeScript — вопросы 29–40 (12)

React — вопросы 41–62 (22)

React Hooks (хуки) — вопросы 63–78 (16)

Redux Toolkit — вопросы 79–90 (12)

Next.js — вопросы 91–100 (10)

Glossary / Словарь терминов

Common phrases / Распространённые фразы

JavaScript

Вопросы 1–28

Q1

What is the difference between var, let, and const?

В чём разница между var, let и const?

ANSWER

var is function-scoped and hoisted with an initial value of undefined. let and const are block-scoped and live in the temporal dead zone until declared. const cannot be reassigned, but for objects and arrays its contents can still be mutated. In modern code we prefer const by default and let when reassignment is needed.

ПЕРЕВОД

var имеет функциональную область видимости и поднимается со значением undefined. let и const имеют блочную область видимости и находятся во временной мёртвой зоне до объявления. const нельзя переприсвоить, но содержимое объектов и массивов всё равно можно изменять. В современном коде по умолчанию используют const, а let — когда нужно переприсваивание.

Example / Пример:

```
var a = 1;       // function-scoped
let b = 2;       // block-scoped, reassignable
const c = 3;     // block-scoped, no reassignment
const arr = [];
arr.push(1);     // OK: mutation, not reassignment
```

Q2

What is hoisting in JavaScript?

Что такое поднятие (hoisting) в JavaScript?

ANSWER

Hoisting is the engine's behavior of moving declarations to the top of their scope during compilation. Function declarations are fully hoisted, so they can be called before they appear. var declarations are hoisted but initialized as undefined. let and const are hoisted too, but accessing them before declaration throws a ReferenceError because of the temporal dead zone.

ПЕРЕВОД

Hoisting — это поведение движка, при котором объявления перемещаются в начало области видимости на этапе компиляции. Объявления функций поднимаются полностью, поэтому их можно вызвать до объявления. var поднимается, но инициализируется значением undefined. let и const тоже поднимаются, но обращение к ним до объявления вызывает ReferenceError из-за временной мёртвой зоны.

Example / Пример:

```
console.log(x); // undefined (var hoisted)
var x = 5;

foo();          // works, function declaration hoisted
function foo() {}
```

Q3

What is a closure and where is it used?

Что такое замыкание и где оно применяется?

ANSWER

A closure is a function that remembers the variables from the lexical scope where it was created, even after that outer function has finished executing. Closures are used for data privacy, function factories, memoization, and keeping state in event handlers or callbacks.

ПЕРЕВОД

Замыкание — это функция, которая запоминает переменные из лексической области, где она была создана, даже после того как внешняя функция завершила работу. Замыкания используют для приватности данных, фабрик функций, мемоизации и хранения состояния в обработчиках событий или колбэках.

Example / Пример:

```
function counter() {
  let count = 0;
  return () => ++count;
}
const inc = counter();
inc(); // 1
inc(); // 2
```

Q4

What is the difference between == and ===?

В чём разница между == и ===?

ANSWER

== is loose equality and performs type coercion before comparing, so values of different types can be considered equal. === is strict equality and compares both value and type without coercion. Best practice is to always use === to avoid unexpected coercion bugs.

ПЕРЕВОД

== — нестрогое равенство, оно приводит типы перед сравнением, поэтому значения разных типов могут считаться равными. === — строгое равенство, сравнивает и значение, и тип без приведения. Хорошая практика — всегда использовать === во избежание неожиданных багов с приведением типов.

Example / Пример:

```
0 == ''     // true  (coercion)
0 === ''    // false
null == undefined  // true
null === undefined // false
```

Q5

Explain the JavaScript event loop.

Объясните событийный цикл (event loop) в JavaScript.

ANSWER

JavaScript is single-threaded. The event loop coordinates the call stack, the task queues, and the Web APIs. Synchronous code runs on the call stack first. When it is empty, the loop processes all microtasks (Promises, queueMicrotask), then one macrotask (setTimeout, events), then microtasks again. This lets async code run without blocking the main thread.

ПЕРЕВОД

JavaScript однопоточный. Событийный цикл координирует стек вызовов, очереди задач и Web API. Сначала на стеке выполняется синхронный код. Когда стек пуст, цикл обрабатывает все микрозадачи (промисы, queueMicrotask), затем одну макрозадачу (setTimeout, события), затем снова микрозадачи. Это позволяет асинхронному коду работать, не блокируя основной поток.

Example / Пример:

```
console.log(1);
setTimeout(() => console.log(2), 0);
Promise.resolve().then(() => console.log(3));
console.log(4);
// Output: 1, 4, 3, 2
```

Q6

What is prototypal inheritance?

Что такое прототипное наследование?

ANSWER

Every JavaScript object has a hidden link to another object called its prototype. When you access a property that doesn't exist on the object, the engine looks up the prototype chain until it finds it or reaches null. This mechanism lets objects share methods and is what class syntax uses under the hood.

ПЕРЕВОД

У каждого объекта в JavaScript есть скрытая ссылка на другой объект — его прототип. При обращении к свойству, которого нет у объекта, движок поднимается по цепочке прототипов, пока не найдёт его или не дойдёт до null. Этот механизм позволяет объектам разделять методы, и именно его под капотом использует синтаксис классов.

Example / Пример:

```
const animal = { eats: true };
const dog = Object.create(animal);
dog.barks = true;
dog.eats;  // true (from prototype)
```

Q7

How does the 'this' keyword work?

Как работает ключевое слово 'this'?

ANSWER

this refers to the execution context and is determined by how a function is called, not where it is defined. In a method call it is the object before the dot; in a plain call it is undefined in strict mode or the global object otherwise; with new it is the new instance. Arrow functions don't have their own this and capture it from the surrounding scope.

ПЕРЕВОД

this указывает на контекст выполнения и определяется тем, как вызвана функция, а не где она объявлена. При вызове метода это объект перед точкой; при обычном вызове — undefined в строгом режиме или глобальный объект иначе; с new — новый экземпляр. Стрелочные функции не имеют своего this и берут его из окружающей области видимости.

Example / Пример:

```
const obj = {
  name: 'A',
  regular() { return this.name; },      // 'A'
  arrow: () => this.name,               // outer this
};
```

Q8

What is the difference between call, apply, and bind?

В чём разница между call, apply и bind?

ANSWER

All three set the value of this for a function. call invokes the function immediately with arguments passed individually. apply does the same but takes arguments as an array. bind does not invoke the function; it returns a new function with this permanently bound, which you can call later.

ПЕРЕВОД

Все три устанавливают значение this для функции. call вызывает функцию сразу, передавая аргументы по отдельности. apply делает то же, но принимает аргументы массивом. bind не вызывает функцию, а возвращает новую функцию с навсегда привязанным this, которую можно вызвать позже.

Example / Пример:

```
function greet(g) { return g + ' ' + this.name; }
const u = { name: 'Anna' };
greet.call(u, 'Hi');       // 'Hi Anna'
greet.apply(u, ['Hi']);    // 'Hi Anna'
const b = greet.bind(u);
b('Hi');                   // 'Hi Anna'
```

Q9

What are Promises and why are they better than callbacks?

Что такое промисы и чем они лучше колбэков?

ANSWER

A Promise is an object representing the eventual result of an asynchronous operation, with states pending, fulfilled, or rejected. Compared to nested callbacks, promises avoid 'callback hell', allow chaining with .then, centralized error handling with .catch, and combinators like Promise.all and Promise.race.

ПЕРЕВОД

Промис — это объект, представляющий будущий результат асинхронной операции, с состояниями pending, fulfilled или rejected. По сравнению с вложенными колбэками промисы избавляют от «ада колбэков», позволяют выстраивать цепочки через .then, централизованно обрабатывать ошибки через .catch и использовать комбинаторы вроде Promise.all и Promise.race.

Example / Пример:

```
fetch('/api')
  .then(res => res.json())
  .then(data => console.log(data))
  .catch(err => console.error(err));
```

Q10

How does async/await work?

Как работает async/await?

ANSWER

async/await is syntactic sugar over Promises that lets you write asynchronous code in a synchronous-looking style. An async function always returns a Promise. await pauses the function until the awaited Promise settles, without blocking the main thread. Errors are handled with try/catch.

ПЕРЕВОД

async/await — это синтаксический сахар над промисами, позволяющий писать асинхронный код в стиле синхронного. async-функция всегда возвращает промис. await приостанавливает выполнение функции до разрешения ожидаемого промиса, не блокируя основной поток. Ошибки обрабатываются через try/catch.

Example / Пример:

```
async function load() {
  try {
    const res = await fetch('/api');
    return await res.json();
  } catch (e) {
    console.error(e);
  }
}
```

Q11

What is the difference between null and undefined?

В чём разница между null и undefined?

ANSWER

undefined means a variable has been declared but not assigned a value, or a missing property/return. null is an intentional assignment representing 'no value'. typeof undefined is 'undefined', while typeof null is 'object' for historical reasons. They are loosely equal (==) but not strictly equal (===).

ПЕРЕВОД

undefined означает, что переменная объявлена, но ей не присвоено значение, либо отсутствует свойство/возврат. null — это намеренное присваивание, означающее «отсутствие значения». typeof undefined — это 'undefined', а typeof null — 'object' по историческим причинам. Они нестрого равны (==), но не строго равны (===).

Example / Пример:

```
let a;          // undefined
let b = null;   // null
a == b;   // true
a === b;  // false
```

Q12

What is the difference between primitive and reference types?

В чём разница между примитивными и ссылочными типами?

ANSWER

Primitives (string, number, boolean, null, undefined, symbol, bigint) are immutable and copied by value. Objects, arrays, and functions are reference types stored on the heap and copied by reference, so two variables can point to the same object. This affects equality checks and how mutations propagate.

ПЕРЕВОД

Примитивы (string, number, boolean, null, undefined, symbol, bigint) неизменяемы и копируются по значению. Объекты, массивы и функции — ссылочные типы, хранятся в куче и копируются по ссылке, поэтому две переменные могут указывать на один и тот же объект. Это влияет на проверки равенства и на то, как распространяются изменения.

Example / Пример:

```
let a = { x: 1 };
let b = a;
b.x = 2;
a.x;        // 2 (same reference)
a === b;    // true
```

Q13

Explain map, filter, and reduce.

Объясните методы map, filter и reduce.

ANSWER

These are non-mutating array methods. map transforms each element and returns a new array of the same length. filter returns a new array with only elements that pass a test. reduce accumulates the array into a single value using an accumulator. They enable a clean, declarative, functional style.

ПЕРЕВОД

Это методы массивов, не изменяющие исходный массив. map преобразует каждый элемент и возвращает новый массив той же длины. filter возвращает новый массив только из элементов, прошедших проверку. reduce сводит массив к одному значению с помощью аккумулятора. Они дают чистый декларативный функциональный стиль.

Example / Пример:

```
const n = [1, 2, 3, 4];
n.map(x => x * 2);          // [2,4,6,8]
n.filter(x => x % 2 === 0); // [2,4]
n.reduce((acc, x) => acc + x, 0); // 10
```

Q14

What are the spread and rest operators?

Что такое операторы расширения (spread) и сбора (rest)?

ANSWER

Both use the ... syntax. Spread expands an iterable or object into individual elements, useful for copying or merging arrays and objects. Rest collects multiple elements into a single array, used for variable function arguments or in destructuring. Context determines which one it is.

ПЕРЕВОД

Оба используют синтаксис ... . Spread разворачивает итерируемое или объект в отдельные элементы — удобно для копирования или объединения массивов и объектов. Rest собирает несколько элементов в один массив — используется для переменного числа аргументов функции или в деструктуризации. Какой это оператор, определяет контекст.

Example / Пример:

```
const merged = [...a, ...b];          // spread
const clone = { ...obj, x: 1 };       // spread
function sum(...nums) {}               // rest
const [first, ...others] = arr;        // rest
```

Q15

What is destructuring?

Что такое деструктуризация?

ANSWER

Destructuring is a syntax for unpacking values from arrays or properties from objects into distinct variables in a concise way. It supports default values, renaming, and nested patterns. It is widely used in function parameters, imports, and React props.

ПЕРЕВОД

Деструктуризация — это синтаксис для лаконичного извлечения значений из массивов или свойств из объектов в отдельные переменные. Она поддерживает значения по умолчанию, переименование и вложенные шаблоны. Широко применяется в параметрах функций, импортах и пропсах React.

Example / Пример:

```
const { name, age = 18 } = user;
const [a, b] = [1, 2];
function f({ id, title }) {}
```

Q16

What is the difference between debounce and throttle?

В чём разница между debounce и throttle?

ANSWER

Both limit how often a function runs. Debounce delays execution until a pause of N ms occurs after the last call, good for search inputs. Throttle guarantees the function runs at most once per interval regardless of how often it's triggered, good for scroll or resize handlers.

ПЕРЕВОД

Оба ограничивают частоту вызова функции. Debounce откладывает выполнение до паузы в N мс после последнего вызова — подходит для полей поиска. Throttle гарантирует, что функция выполнится не чаще одного раза за интервал, как бы часто её ни вызывали — подходит для обработчиков прокрутки или ресайза.

Example / Пример:

```
function debounce(fn, ms) {
  let t;
  return (...args) => {
    clearTimeout(t);
    t = setTimeout(() => fn(...args), ms);
  };
}
```

Q17

What is the difference between a shallow copy and a deep copy?

В чём разница между поверхностным и глубоким копированием?

ANSWER

A shallow copy duplicates only the top level; nested objects are still shared by reference. A deep copy recursively clones everything, so the copy is fully independent. Shallow copies use spread or Object.assign; deep copies use structuredClone or libraries like lodash.cloneDeep.

ПЕРЕВОД

Поверхностная копия дублирует только верхний уровень; вложенные объекты по-прежнему разделяются по ссылке. Глубокая копия рекурсивно клонирует всё, поэтому копия полностью независима. Для поверхностной копии используют spread или Object.assign; для глубокой — structuredClone или библиотеки вроде lodash.cloneDeep.

Example / Пример:

```
const shallow = { ...obj };
const deep = structuredClone(obj);
```

Q18

What is event delegation?

Что такое делегирование событий?

ANSWER

Event delegation is a pattern where you attach a single event listener to a parent element instead of many listeners on children, relying on event bubbling. Inside the handler you check event.target to determine which child triggered it. It improves performance and works for dynamically added elements.

ПЕРЕВОД

Делегирование событий — это паттерн, при котором вы вешаете один обработчик на родительский элемент вместо множества обработчиков на дочерние, опираясь на всплытие событий. Внутри обработчика вы проверяете event.target, чтобы определить, какой дочерний элемент его вызвал. Это улучшает производительность и работает для динамически добавленных элементов.

Example / Пример:

```
list.addEventListener('click', (e) => {
  if (e.target.matches('li')) {
    console.log(e.target.textContent);
  }
});
```

Q19

What are higher-order functions?

Что такое функции высшего порядка?

ANSWER

A higher-order function is a function that takes another function as an argument, returns a function, or both. They enable abstraction and code reuse. Array methods like map and filter are higher-order, and so are decorators, debounce, and middleware.

ПЕРЕВОД

Функция высшего порядка — это функция, которая принимает другую функцию в качестве аргумента, возвращает функцию или и то, и другое. Они позволяют абстрагироваться и переиспользовать код. Методы массивов вроде map и filter являются функциями высшего порядка, как и декораторы, debounce и middleware.

Example / Пример:

```
const withLog = (fn) => (...args) => {
  console.log('called');
  return fn(...args);
};
```

Q20

What is the difference between microtasks and macrotasks?

В чём разница между микрозадачами и макрозадачами?

ANSWER

Microtasks (Promise callbacks, queueMicrotask, MutationObserver) run immediately after the current synchronous code finishes and before any rendering or macrotask. Macrotasks (setTimeout, setInterval, I/O, events) run one per event-loop iteration. The microtask queue is fully drained before the next macrotask.

ПЕРЕВОД

Микрозадачи (колбэки промисов, queueMicrotask, MutationObserver) выполняются сразу после завершения текущего синхронного кода и до рендеринга или любой макрозадачи. Макрозадачи (setTimeout, setInterval, ввод-вывод, события) выполняются по одной за итерацию цикла событий. Очередь микрозадач полностью опустошается перед следующей макрозадачей.

Example / Пример:

```
setTimeout(() => console.log('macro'));
Promise.resolve().then(() => console.log('micro'));
// micro logs before macro
```

Q21

What is currying?

Что такое каррирование?

ANSWER

Currying transforms a function that takes multiple arguments into a sequence of functions that each take one argument. It enables partial application, where you fix some arguments and reuse the resulting function. It's common in functional programming and libraries like Ramda.

ПЕРЕВОД

Каррирование преобразует функцию, принимающую несколько аргументов, в последовательность функций, каждая из которых принимает один аргумент. Это позволяет делать частичное применение — зафиксировать часть аргументов и переиспользовать полученную функцию. Часто встречается в функциональном программировании и библиотеках вроде Ramda.

Example / Пример:

```
const add = a => b => c => a + b + c;
add(1)(2)(3);   // 6
const add5 = add(5);
```

Q22

What are optional chaining and nullish coalescing?

Что такое опциональная цепочка и оператор нулевого слияния?

ANSWER

Optional chaining (?.) safely accesses nested properties and returns undefined instead of throwing if a link in the chain is null or undefined. Nullish coalescing (??) returns the right side only when the left side is null or undefined, unlike || which also reacts to falsy values like 0 or ''.

ПЕРЕВОД

Опциональная цепочка (?.) безопасно обращается к вложенным свойствам и возвращает undefined вместо ошибки, если звено цепочки равно null или undefined. Оператор нулевого слияния (??) возвращает правую часть только когда левая равна null или undefined, в отличие от ||, который реагирует и на ложные значения вроде 0 или ''.

Example / Пример:

```
user?.address?.city;        // undefined if missing
const port = config.port ?? 3000; // 0 stays 0
```

Q23

What is the temporal dead zone?

Что такое временная мёртвая зона?

ANSWER

The temporal dead zone (TDZ) is the period between entering a scope and the actual declaration of a let or const variable. The variable exists but cannot be accessed; doing so throws a ReferenceError. This helps catch bugs from using variables before they're initialized.

ПЕРЕВОД

Временная мёртвая зона (TDZ) — это промежуток между входом в область видимости и фактическим объявлением переменной let или const. Переменная существует, но обращаться к ней нельзя; попытка вызывает ReferenceError. Это помогает ловить ошибки использования переменных до их инициализации.

Example / Пример:

```
console.log(x); // ReferenceError (TDZ)
let x = 5;
```

Q24

How does garbage collection work in JavaScript?

Как работает сборка мусора в JavaScript?

ANSWER

JavaScript automatically frees memory that is no longer reachable. The main algorithm is mark-and-sweep: starting from roots like the global object, it marks all reachable values and sweeps away the rest. Memory leaks happen when references are accidentally kept alive, for example in closures, detached DOM nodes, or forgotten timers.

ПЕРЕВОД

JavaScript автоматически освобождает память, которая больше недостижима. Основной алгоритм — пометка и очистка (mark-and-sweep): начиная с корней вроде глобального объекта, он помечает все достижимые значения и удаляет остальные. Утечки памяти возникают, когда ссылки случайно остаются живыми — например, в замыканиях, отсоединённых DOM-узлах или забытых таймерах.

Example / Пример:

```
// Leak: timer keeps closure alive
setInterval(() => use(bigData), 1000);
// Fix: clearInterval when no longer needed
```

Q25

What is the difference between synchronous and asynchronous code?

В чём разница между синхронным и асинхронным кодом?

ANSWER

Synchronous code runs line by line and blocks the thread until each operation completes. Asynchronous code starts an operation and continues without waiting, handling the result later via callbacks, promises, or async/await. Async is essential for I/O like network requests so the UI stays responsive.

ПЕРЕВОД

Синхронный код выполняется строка за строкой и блокирует поток, пока каждая операция не завершится. Асинхронный код запускает операцию и продолжает работу не дожидаясь, обрабатывая результат позже через колбэки, промисы или async/await. Асинхронность необходима для ввода-вывода вроде сетевых запросов, чтобы интерфейс оставался отзывчивым.

Example / Пример:

```
console.log('start');
setTimeout(() => console.log('async'), 0);
console.log('end');
// start, end, async
```

Q26

What is a generator function?

Что такое функция-генератор?

ANSWER

A generator function, declared with function*, can pause and resume execution using the yield keyword. Calling it returns an iterator whose next() runs until the next yield. Generators are useful for lazy sequences, custom iterators, and were historically used for async flow control.

ПЕРЕВОД

Функция-генератор, объявляемая через function*, может приостанавливать и возобновлять выполнение с помощью ключевого слова yield. Её вызов возвращает итератор, чей next() выполняется до следующего yield. Генераторы полезны для ленивых последовательностей, пользовательских итераторов и исторически применялись для управления асинхронным потоком.

Example / Пример:

```
function* gen() {
  yield 1;
  yield 2;
}
const it = gen();
it.next(); // { value: 1, done: false }
```

Q27

What are WeakMap and WeakSet?

Что такое WeakMap и WeakSet?

ANSWER

WeakMap and WeakSet hold weak references to their keys (objects only). If there are no other references to a key, it can be garbage collected, and the entry disappears automatically. They are not iterable and have no size. They're useful for caching or storing private metadata tied to an object's lifetime.

ПЕРЕВОД

WeakMap и WeakSet хранят слабые ссылки на свои ключи (только объекты). Если других ссылок на ключ нет, он может быть удалён сборщиком мусора, и запись исчезнет автоматически. Они не итерируемы и не имеют size. Полезны для кэширования или хранения приватных метаданных, привязанных к времени жизни объекта.

Example / Пример:

```
const cache = new WeakMap();
cache.set(obj, computedValue);
// entry auto-removed when obj is gone
```

Q28

How do you handle errors in asynchronous code?

Как обрабатывать ошибки в асинхронном коде?

ANSWER

With promises you use .catch to handle rejections; with async/await you wrap awaits in try/catch. For multiple parallel operations, Promise.allSettled lets you inspect each result. Always handle rejections, since unhandled promise rejections can crash Node processes or be silently lost in the browser.

ПЕРЕВОД

С промисами для обработки отклонений используют .catch; с async/await оборачивают await в try/catch. Для нескольких параллельных операций Promise.allSettled позволяет проверить каждый результат. Всегда обрабатывайте отклонения, поскольку необработанные отклонения промисов могут уронить процесс Node или незаметно потеряться в браузере.

Example / Пример:

```
async function safe() {
  try {
    await risky();
  } catch (e) {
    log(e);
  } finally {
    cleanup();
  }
}
```

TypeScript

Вопросы 29–40

Q29

What is TypeScript and why use it?

Что такое TypeScript и зачем его использовать?

ANSWER

TypeScript is a typed superset of JavaScript that compiles to plain JavaScript. It adds static type checking, interfaces, generics, and modern tooling. The benefits are catching errors at compile time, better autocompletion and refactoring, self-documenting code, and safer large-scale codebases.

ПЕРЕВОД

TypeScript — это типизированное надмножество JavaScript, которое компилируется в обычный JavaScript. Он добавляет статическую проверку типов, интерфейсы, дженерики и современный инструментарий. Преимущества — обнаружение ошибок на этапе компиляции, лучшее автодополнение и рефакторинг, самодокументируемый код и более безопасные крупные кодовые базы.

Example / Пример:

```
function sum(a: number, b: number): number {
  return a + b;
}
sum(1, '2'); // compile-time error
```

Q30

What is the difference between interface and type?

В чём разница между interface и type?

ANSWER

Both describe the shape of data. interface is extendable and supports declaration merging, and is preferred for object and class contracts. type is more flexible: it can describe unions, intersections, primitives, tuples, and mapped types. In practice many teams use interface for objects and type for everything else.

ПЕРЕВОД

Оба описывают форму данных. interface расширяемый и поддерживает слияние объявлений, его предпочитают для контрактов объектов и классов. type более гибкий: он может описывать объединения, пересечения, примитивы, кортежи и сопоставленные типы. На практике многие команды используют interface для объектов, а type для всего остального.

Example / Пример:

```
interface User { id: number; name: string; }
type ID = string | number;
type Point = { x: number } & { y: number };
```

Q31

What are generics?

Что такое дженерики (обобщения)?

ANSWER

Generics let you write reusable components and functions that work with multiple types while preserving type safety. A type parameter acts as a placeholder filled in at usage. They are everywhere in TypeScript: arrays, promises, React hooks, and utility types all use generics.

ПЕРЕВОД

Дженерики позволяют писать переиспользуемые компоненты и функции, работающие с разными типами и сохраняющие типобезопасность. Параметр типа выступает заполнителем, который подставляется при использовании. Они повсюду в TypeScript: массивы, промисы, хуки React и служебные типы используют дженерики.

Example / Пример:

```
function identity<T>(value: T): T {
  return value;
}
identity<string>('hi');
identity(42); // T inferred as number
```

Q32

What are enums in TypeScript?

Что такое перечисления (enum) в TypeScript?

ANSWER

An enum is a named set of related constants, making intent clearer than magic numbers or strings. There are numeric and string enums. Note that numeric enums generate extra runtime code; many teams now prefer string enums or 'as const' object literals for a lighter footprint.

ПЕРЕВОД

Enum — это именованный набор связанных констант, делающий смысл понятнее, чем «магические» числа или строки. Бывают числовые и строковые перечисления. Учтите, что числовые enum генерируют дополнительный код в рантайме; многие команды теперь предпочитают строковые enum или объекты с 'as const' ради меньшего размера.

Example / Пример:

```
enum Status { Active = 'ACTIVE', Done = 'DONE' }
const s: Status = Status.Active;
```

Q33

What is the difference between unknown and any?

В чём разница между unknown и any?

ANSWER

any disables type checking entirely and lets you do anything, sacrificing safety. unknown is the type-safe counterpart: you can assign anything to it, but you must narrow it with a type check or assertion before using it. Prefer unknown over any to keep type safety while accepting arbitrary values.

ПЕРЕВОД

any полностью отключает проверку типов и позволяет делать что угодно, жертвуя безопасностью. unknown — это его типобезопасный аналог: ему можно присвоить что угодно, но перед использованием нужно сузить тип проверкой или утверждением. Предпочитайте unknown вместо any, чтобы сохранить типобезопасность, принимая произвольные значения.

Example / Пример:

```
let a: any = 5;  a.foo();        // no error, unsafe
let u: unknown = 5;
if (typeof u === 'number') u.toFixed(); // safe
```

Q34

What are union and intersection types?

Что такое объединения и пересечения типов?

ANSWER

A union type (A | B) means a value can be one of several types, and you usually narrow it before use. An intersection type (A & B) combines multiple types into one that has all their members. Unions express 'or', intersections express 'and'.

ПЕРЕВОД

Объединение (A | B) означает, что значение может быть одним из нескольких типов, и обычно его сужают перед использованием. Пересечение (A & B) объединяет несколько типов в один, содержащий все их члены. Объединение выражает «или», пересечение — «и».

Example / Пример:

```
type Result = 'ok' | 'error';
type Admin = User & { role: 'admin' };
```

Q35

What are utility types like Partial, Pick, Omit, and Record?

Что такое служебные типы Partial, Pick, Omit и Record?

ANSWER

Utility types transform existing types. Partial<T> makes all properties optional. Pick<T, K> selects a subset of keys. Omit<T, K> removes keys. Record<K, V> builds an object type with given keys and value type. They reduce duplication and keep types in sync with their source.

ПЕРЕВОД

Служебные типы преобразуют существующие типы. Partial<T> делает все свойства необязательными. Pick<T, K> выбирает подмножество ключей. Omit<T, K> удаляет ключи. Record<K, V> строит тип объекта с заданными ключами и типом значений. Они уменьшают дублирование и держат типы синхронизированными с источником.

Example / Пример:

```
type U = { id: number; name: string; age: number };
type Draft = Partial<U>;
type Preview = Pick<U, 'id' | 'name'>;
type NoId = Omit<U, 'id'>;
type Map = Record<string, number>;
```

Q36

What is type assertion?

Что такое утверждение типа (type assertion)?

ANSWER

Type assertion tells the compiler to treat a value as a specific type when you know more than the type system does, using the 'as' keyword. It performs no runtime conversion or checking, so it can hide bugs if misused. Prefer type guards over assertions when possible.

ПЕРЕВОД

Утверждение типа сообщает компилятору рассматривать значение как определённый тип, когда вы знаете больше системы типов, с помощью ключевого слова 'as'. Оно не выполняет преобразования или проверки в рантайме, поэтому при неправильном применении может скрывать ошибки. По возможности предпочитайте сужающие проверки (type guards) утверждениям.

Example / Пример:

```
const el = document.getElementById('x') as HTMLInputElement;
const value = data as unknown as User;
```

Q37

What is the never type?

Что такое тип never?

ANSWER

never represents values that never occur: a function that always throws or has an infinite loop, or a branch that should be unreachable. It's commonly used for exhaustiveness checking in switch statements, so adding a new union member produces a compile-time error if you forget to handle it.

ПЕРЕВОД

never представляет значения, которые никогда не возникают: функция, которая всегда выбрасывает исключение или зацикливается, либо недостижимая ветка. Часто используется для проверки полноты (exhaustiveness) в switch, чтобы добавление нового члена объединения вызывало ошибку компиляции, если вы забыли его обработать.

Example / Пример:

```
function assertNever(x: never): never {
  throw new Error('Unexpected: ' + x);
}
```

Q38

What are type guards?

Что такое сужающие проверки типов (type guards)?

ANSWER

Type guards narrow a broader type to a more specific one within a block. Built-in guards include typeof, instanceof, and the 'in' operator. You can also write custom guards using a return type of 'param is Type'. They let TypeScript safely access type-specific members.

ПЕРЕВОД

Type guards сужают более широкий тип до более конкретного внутри блока. Встроенные проверки — typeof, instanceof и оператор 'in'. Можно писать и собственные проверки с типом возврата 'param is Type'. Они позволяют TypeScript безопасно обращаться к специфичным для типа членам.

Example / Пример:

```
function isString(x: unknown): x is string {
  return typeof x === 'string';
}
if (isString(v)) v.toUpperCase();
```

Q39

What do readonly and 'as const' do?

Что делают readonly и 'as const'?

ANSWER

readonly marks a property or array as immutable at compile time, preventing reassignment. 'as const' creates a deeply readonly literal type, narrowing values to their exact literals instead of widening them. Both improve safety; note they are compile-time only and don't freeze objects at runtime.

ПЕРЕВОД

readonly помечает свойство или массив как неизменяемые на этапе компиляции, запрещая переприсваивание. 'as const' создаёт глубоко readonly литеральный тип, сужая значения до их точных литералов вместо расширения. Оба повышают безопасность; учтите, что они работают только на этапе компиляции и не замораживают объекты в рантайме.

Example / Пример:

```
const cfg = { mode: 'dark' } as const;
// cfg.mode has type 'dark', not string
interface P { readonly id: number; }
```

Q40

How do you type functions and what are overloads?

Как типизировать функции и что такое перегрузки?

ANSWER

You annotate parameters and return type, and can describe a function with a type or interface. Function overloads provide multiple call signatures for one implementation, letting the return type depend on the argument types. The implementation signature must be compatible with all overloads.

ПЕРЕВОД

Вы аннотируете параметры и тип возврата и можете описать функцию через type или interface. Перегрузки функций задают несколько сигнатур вызова для одной реализации, позволяя типу возврата зависеть от типов аргументов. Сигнатура реализации должна быть совместима со всеми перегрузками.

Example / Пример:

```
function len(x: string): number;
function len(x: any[]): number;
function len(x: any): number { return x.length; }
```

React

Вопросы 41–62

Q41

What is React and what is the virtual DOM?

Что такое React и что такое виртуальный DOM?

ANSWER

React is a declarative library for building user interfaces from reusable components. The virtual DOM is a lightweight in-memory representation of the UI. On state changes React builds a new virtual tree, diffs it against the previous one, and applies only the minimal real DOM updates, which makes rendering efficient.

ПЕРЕВОД

React — это декларативная библиотека для построения пользовательских интерфейсов из переиспользуемых компонентов. Виртуальный DOM — это лёгкое представление интерфейса в памяти. При изменении состояния React строит новое виртуальное дерево, сравнивает его с предыдущим и применяет только минимальные обновления реального DOM, что делает рендеринг эффективным.

Q42

What is JSX?

Что такое JSX?

ANSWER

JSX is a syntax extension that lets you write HTML-like markup inside JavaScript. It is not valid JS by itself; tools like Babel compile it into React.createElement calls (or the modern jsx runtime). JSX must return a single root, uses className instead of class, and embeds expressions in curly braces.

ПЕРЕВОД

JSX — это синтаксическое расширение, позволяющее писать HTML-подобную разметку внутри JavaScript. Сам по себе это не валидный JS; инструменты вроде Babel компилируют его в вызовы React.createElement (или современный jsx-runtime). JSX должен возвращать один корневой элемент, использует className вместо class и встраивает выражения в фигурных скобках.

Example / Пример:

```
const el = <h1 className="title">{user.name}</h1>;
// compiles to React.createElement('h1', ...)
```

Q43

What is the difference between function and class components?

В чём разница между функциональными и классовыми компонентами?

ANSWER

Class components use ES6 classes, lifecycle methods, and this.state. Function components are plain functions that use Hooks for state and side effects. Function components with Hooks are now the standard: they are shorter, easier to test, and avoid 'this' binding issues.

ПЕРЕВОД

Классовые компоненты используют ES6-классы, методы жизненного цикла и this.state. Функциональные компоненты — это обычные функции, использующие хуки для состояния и побочных эффектов. Функциональные компоненты с хуками теперь стандарт: они короче, проще тестируются и избавляют от проблем с привязкой 'this'.

Example / Пример:

```
function Hello({ name }) {
  return <p>Hi {name}</p>;
}
```

Q44

What is the difference between props and state?

В чём разница между props и state?

ANSWER

Props are read-only inputs passed from a parent to a child, making components configurable and reusable. State is data managed internally by a component that can change over time and triggers re-renders. Props flow down; state is local and owned by the component (or lifted to a shared parent).

ПЕРЕВОД

Props — это входные данные только для чтения, передаваемые от родителя к ребёнку, делающие компоненты настраиваемыми и переиспользуемыми. State — это данные, управляемые внутри компонента, которые могут меняться со временем и вызывают повторный рендер. Props текут вниз; state локален и принадлежит компоненту (или поднимается к общему родителю).

Example / Пример:

```
<Avatar size={40} />          // props
const [open, setOpen] = useState(false); // state
```

Q45

Why does React need keys in lists?

Зачем React нужны ключи (keys) в списках?

ANSWER

Keys give each list item a stable identity so React can match elements between renders and update, reorder, or remove them efficiently. Keys should be unique and stable, ideally a data id. Without proper keys React may re-render incorrectly or lose component state.

ПЕРЕВОД

Ключи дают каждому элементу списка стабильную идентичность, чтобы React мог сопоставлять элементы между рендерами и эффективно обновлять, переставлять или удалять их. Ключи должны быть уникальными и стабильными, в идеале — id из данных. Без корректных ключей React может рендерить неправильно или терять состояние компонентов.

Example / Пример:

```
items.map(item => <Row key={item.id} {...item} />)
```

Q46

What is the difference between controlled and uncontrolled components?

В чём разница между управляемыми и неуправляемыми компонентами?

ANSWER

A controlled component has its form value driven by React state and updated via onChange, giving you full control and validation. An uncontrolled component keeps its value in the DOM and you read it with a ref. Controlled is the default recommended approach for forms.

ПЕРЕВОД

У управляемого компонента значение формы управляется состоянием React и обновляется через onChange, что даёт полный контроль и валидацию. Неуправляемый компонент хранит значение в DOM, и вы читаете его через ref. Управляемый подход рекомендуется по умолчанию для форм.

Example / Пример:

```
<input value={text} onChange={e => setText(e.target.value)} /> // controlled
<input ref={inputRef} /> // uncontrolled
```

Q47

What does 'lifting state up' mean?

Что значит «поднятие состояния вверх» (lifting state up)?

ANSWER

When two components need to share or sync data, you move the state to their closest common parent and pass it down via props, along with callbacks to update it. This keeps a single source of truth and avoids duplicated, out-of-sync state.

ПЕРЕВОД

Когда двум компонентам нужно разделять или синхронизировать данные, состояние перемещают в их ближайшего общего родителя и передают вниз через props вместе с колбэками для его обновления. Это сохраняет единый источник истины и избегает дублированного, рассинхронизированного состояния.

Example / Пример:

```
function Parent() {
  const [v, setV] = useState('');
  return <><Input value={v} onChange={setV} /><Preview value={v} /></>;
}
```

Q48

How do you do conditional rendering in React?

Как делать условный рендеринг в React?

ANSWER

You render different output based on a condition using ternary operators, logical && for show/hide, early returns, or variables that hold JSX. Be careful with && and numeric values like 0, since 0 is falsy but still renders; use a boolean or ternary to avoid showing a stray 0.

ПЕРЕВОД

Вы выводите разное содержимое в зависимости от условия с помощью тернарного оператора, логического && для показа/скрытия, ранних возвратов или переменных с JSX. Будьте осторожны с && и числами вроде 0: 0 ложно, но всё равно отрендерится; используйте булево значение или тернар, чтобы случайно не показать 0.

Example / Пример:

```
{isLoading ? <Spinner /> : <List items={items} />}
{count > 0 && <Badge count={count} />}
```

Q49

What is reconciliation in React?

Что такое согласование (reconciliation) в React?

ANSWER

Reconciliation is the diffing process React uses to figure out what changed between the previous and new virtual DOM trees. It uses heuristics: different element types produce a full subtree replacement, and keys help match children. The result is a minimal set of real DOM mutations.

ПЕРЕВОД

Согласование — это процесс сравнения, который React использует, чтобы понять, что изменилось между предыдущим и новым деревьями виртуального DOM. Он применяет эвристики: разные типы элементов приводят к полной замене поддерева, а ключи помогают сопоставлять дочерние элементы. Результат — минимальный набор изменений реального DOM.

Q50

What is React.memo and when should you use it?

Что такое React.memo и когда его использовать?

ANSWER

React.memo is a higher-order component that memoizes a function component, skipping re-render when its props are shallowly equal to the previous render. Use it for expensive components that re-render often with the same props. Overusing it adds comparison overhead, so apply it where it measurably helps.

ПЕРЕВОД

React.memo — это компонент высшего порядка, который мемоизирует функциональный компонент, пропуская повторный рендер, если его props поверхностно равны предыдущему рендеру. Используйте его для «дорогих» компонентов, которые часто перерисовываются с одними и теми же props. Чрезмерное применение добавляет накладные расходы на сравнение, поэтому применяйте там, где это измеримо помогает.

Example / Пример:

```
const Row = React.memo(function Row({ data }) {
  return <div>{data.title}</div>;
});
```

Q51

What is the Context API?

Что такое Context API?

ANSWER

Context provides a way to pass data through the component tree without manually threading props at every level, solving 'prop drilling'. You create a context, wrap part of the tree in a Provider, and read it with useContext. It's best for low-frequency global data like theme, locale, or auth.

ПЕРЕВОД

Context даёт способ передавать данные по дереву компонентов без ручной передачи props на каждом уровне, решая проблему «prop drilling». Вы создаёте контекст, оборачиваете часть дерева в Provider и читаете его через useContext. Лучше всего подходит для редко меняющихся глобальных данных вроде темы, локали или авторизации.

Example / Пример:

```
const Theme = createContext('light');
<Theme.Provider value="dark"><App /></Theme.Provider>
const theme = useContext(Theme);
```

Q52

What are error boundaries?

Что такое границы ошибок (error boundaries)?

ANSWER

Error boundaries are components that catch JavaScript errors in their child tree during rendering, display a fallback UI, and prevent the whole app from crashing. They are implemented as class components with componentDidCatch or getDerivedStateFromError. They don't catch errors in event handlers or async code.

ПЕРЕВОД

Границы ошибок — это компоненты, которые перехватывают ошибки JavaScript в дочернем дереве во время рендеринга, показывают запасной интерфейс и не дают всему приложению упасть. Реализуются как классовые компоненты с componentDidCatch или getDerivedStateFromError. Они не перехватывают ошибки в обработчиках событий или асинхронном коде.

Example / Пример:

```
class Boundary extends React.Component {
  static getDerivedStateFromError() { return { hasError: true }; }
  render() { return this.state.hasError ? <Fallback /> : this.props.children; }
}
```

Q53

What are Fragments?

Что такое фрагменты (Fragments)?

ANSWER

A Fragment lets you group multiple children without adding an extra DOM node, since a component must return a single root. You can write it as <React.Fragment> or the shorthand <>...</>. Use the full form with a key when rendering fragments in a list.

ПЕРЕВОД

Фрагмент позволяет группировать несколько дочерних элементов без добавления лишнего DOM-узла, ведь компонент должен возвращать один корень. Можно писать <React.Fragment> или сокращённо <>...</>. Используйте полную форму с key, когда рендерите фрагменты в списке.

Example / Пример:

```
return (
  <>
    <th>Name</th>
    <th>Age</th>
  </>
);
```

Q54

What are refs and what is forwardRef?

Что такое refs и что такое forwardRef?

ANSWER

A ref gives you direct access to a DOM node or persists a mutable value across renders without causing re-renders. forwardRef lets a parent pass a ref through a component down to a child DOM element, which is useful for reusable input components. (In React 19 ref can be a regular prop.)

ПЕРЕВОД

Ref даёт прямой доступ к DOM-узлу или хранит изменяемое значение между рендерами, не вызывая повторного рендера. forwardRef позволяет родителю передать ref через компонент к дочернему DOM-элементу, что полезно для переиспользуемых компонентов ввода. (В React 19 ref может быть обычным пропом.)

Example / Пример:

```
const Input = forwardRef((props, ref) => <input ref={ref} {...props} />);
```

Q55

What is a higher-order component (HOC)?

Что такое компонент высшего порядка (HOC)?

ANSWER

A HOC is a function that takes a component and returns a new component with added behavior, used for reusing cross-cutting logic like authentication or data loading. Hooks have largely replaced HOCs for sharing logic, but you still see them in older codebases and some libraries.

ПЕРЕВОД

HOC — это функция, которая принимает компонент и возвращает новый компонент с добавленным поведением; используется для переиспользования сквозной логики вроде авторизации или загрузки данных. Хуки в основном заменили HOC для разделения логики, но их всё ещё можно встретить в старых проектах и некоторых библиотеках.

Example / Пример:

```
function withAuth(Component) {
  return (props) => isLoggedIn ? <Component {...props} /> : <Login />;
}
```

Q56

What is the render props pattern?

Что такое паттерн render props?

ANSWER

Render props is a technique where a component takes a function as a prop (often children) and calls it to decide what to render, sharing logic while leaving the UI to the consumer. Like HOCs, it's now often replaced by custom hooks, but it's still useful for certain flexible APIs.

ПЕРЕВОД

Render props — это приём, при котором компонент принимает функцию в качестве пропа (часто children) и вызывает её, чтобы решить, что отрендерить, разделяя логику и оставляя интерфейс на усмотрение потребителя. Как и HOC, его теперь часто заменяют кастомными хуками, но он по-прежнему полезен для некоторых гибких API.

Example / Пример:

```
<Mouse render={({ x, y }) => <p>{x}, {y}</p>} />
```

Q57

What is lazy loading and code splitting in React?

Что такое ленивая загрузка и разделение кода в React?

ANSWER

Code splitting breaks the bundle into smaller chunks loaded on demand, improving initial load time. React.lazy dynamically imports a component, and Suspense shows a fallback while it loads. This is commonly applied at the route level to load each page only when visited.

ПЕРЕВОД

Разделение кода разбивает бандл на меньшие части, загружаемые по требованию, улучшая время первоначальной загрузки. React.lazy динамически импортирует компонент, а Suspense показывает запасной интерфейс во время загрузки. Чаще всего применяется на уровне маршрутов, чтобы загружать каждую страницу только при переходе на неё.

Example / Пример:

```
const Page = React.lazy(() => import('./Page'));
<Suspense fallback={<Spinner />}><Page /></Suspense>
```

Q58

What are synthetic events in React?

Что такое синтетические события в React?

ANSWER

React wraps native browser events in a cross-browser wrapper called SyntheticEvent, providing a consistent API. React attaches a single delegated listener at the root rather than per element. You access the native event via event.nativeEvent if needed.

ПЕРЕВОД

React оборачивает нативные события браузера в кроссбраузерную обёртку SyntheticEvent, предоставляя единообразный API. React навешивает один делегированный обработчик в корне вместо отдельного на каждый элемент. При необходимости нативное событие доступно через event.nativeEvent.

Example / Пример:

```
<button onClick={(e) => { e.preventDefault(); }}>Save</button>
```

Q59

Why is using array index as a key problematic?

Почему использовать индекс массива в качестве ключа проблематично?

ANSWER

Using the index as a key breaks React's ability to track items when the list is reordered, filtered, or items are inserted/removed, because the same index maps to different data. This can cause wrong updates, lost input state, or visual bugs. Use a stable unique id instead.

ПЕРЕВОД

Использование индекса в качестве ключа ломает способность React отслеживать элементы при переупорядочивании, фильтрации или вставке/удалении, потому что один индекс начинает соответствовать другим данным. Это может вызвать неверные обновления, потерю состояния ввода или визуальные баги. Используйте стабильный уникальный id.

Example / Пример:

```
// Bad:  items.map((it, i) => <Row key={i} />)
// Good: items.map(it => <Row key={it.id} />)
```

Q60

How do you optimize performance in a React application?

Как оптимизировать производительность React-приложения?

ANSWER

Common techniques: memoize components with React.memo and values/functions with useMemo/useCallback, split code with lazy/Suspense, virtualize long lists, use stable keys, avoid creating new objects in render, and lift state appropriately to limit re-renders. Always profile first with React DevTools before optimizing.

ПЕРЕВОД

Распространённые приёмы: мемоизация компонентов через React.memo и значений/функций через useMemo/useCallback, разделение кода через lazy/Suspense, виртуализация длинных списков, стабильные ключи, отказ от создания новых объектов в рендере и грамотное размещение состояния для ограничения повторных рендеров. Всегда сначала профилируйте через React DevTools, прежде чем оптимизировать.

Q61

What is StrictMode?

Что такое StrictMode?

ANSWER

StrictMode is a development-only wrapper that helps surface potential problems: it warns about deprecated APIs and unsafe lifecycles, and intentionally double-invokes certain functions like render and effects to reveal impure logic or missing cleanups. It has no effect in production and renders no visible UI.

ПЕРЕВОД

StrictMode — это обёртка, работающая только в режиме разработки, которая помогает выявлять потенциальные проблемы: предупреждает об устаревших API и небезопасных методах жизненного цикла и намеренно вызывает дважды некоторые функции вроде render и эффектов, чтобы обнаружить нечистую логику или отсутствующую очистку. В продакшене не действует и не рендерит видимого интерфейса.

Example / Пример:

```
<React.StrictMode><App /></React.StrictMode>
```

Q62

Are state updates synchronous, and what is batching?

Синхронны ли обновления состояния и что такое батчинг?

ANSWER

State updates are asynchronous: calling a setter schedules a re-render rather than changing the value immediately, so reading state right after won't reflect the update. React also batches multiple updates within the same event into a single re-render for performance. Since React 18, batching also applies inside promises, timeouts, and native event handlers (automatic batching).

ПЕРЕВОД

Обновления состояния асинхронны: вызов сеттера планирует повторный рендер, а не меняет значение сразу, поэтому чтение состояния сразу после не отразит обновление. React также группирует (батчит) несколько обновлений внутри одного события в один повторный рендер ради производительности. Начиная с React 18, батчинг работает и внутри промисов, таймаутов и нативных обработчиков (автоматический батчинг).

Example / Пример:

```
setCount(c + 1);
setCount(c + 1);
// batched: one re-render; use updater for cumulative updates
```

React Hooks (хуки)

Вопросы 63–78

Q63

What are the rules of hooks?

Каковы правила хуков?

ANSWER

Two main rules: only call hooks at the top level of a function component or custom hook, never inside loops, conditions, or nested functions; and only call them from React functions, not regular JS functions. These rules ensure hooks are called in the same order every render so React can track their state correctly.

ПЕРЕВОД

Два главных правила: вызывайте хуки только на верхнем уровне функционального компонента или кастомного хука, никогда внутри циклов, условий или вложенных функций; и вызывайте их только из React-функций, а не из обычных JS-функций. Эти правила гарантируют, что хуки вызываются в одном и том же порядке при каждом рендере, чтобы React мог корректно отслеживать их состояние.

Example / Пример:

```
// Bad: if (cond) useState();
// Good: const [s, setS] = useState(); // top level
```

Q64

How does useState work?

Как работает useState?

ANSWER

useState declares a piece of local state and returns the current value and a setter. Calling the setter schedules a re-render with the new value. State updates are batched and asynchronous, so to update based on the previous value you should pass an updater function rather than reading the current state.

ПЕРЕВОД

useState объявляет фрагмент локального состояния и возвращает текущее значение и функцию-сеттер. Вызов сеттера планирует повторный рендер с новым значением. Обновления состояния группируются и асинхронны, поэтому для обновления на основе предыдущего значения следует передавать функцию-обновитель, а не читать текущее состояние.

Example / Пример:

```
const [count, setCount] = useState(0);
setCount(prev => prev + 1); // safe update
```

Q65

What does useEffect do and how does cleanup work?

Что делает useEffect и как работает очистка?

ANSWER

useEffect runs side effects (data fetching, subscriptions, timers) after render. If you return a function, React runs it as cleanup before the next effect run and on unmount, which is where you cancel subscriptions or clear timers. The dependency array controls when the effect re-runs.

ПЕРЕВОД

useEffect выполняет побочные эффекты (загрузку данных, подписки, таймеры) после рендера. Если вернуть функцию, React выполнит её как очистку перед следующим запуском эффекта и при размонтировании — там вы отменяете подписки или очищаете таймеры. Массив зависимостей определяет, когда эффект перезапускается.

Example / Пример:

```
useEffect(() => {
  const id = setInterval(tick, 1000);
  return () => clearInterval(id); // cleanup
}, []);
```

Q66

What does the dependency array of useEffect control?

Что контролирует массив зависимостей useEffect?

ANSWER

The dependency array tells React when to re-run the effect. An empty array runs it once after mount. Listing values runs it whenever any of them change. Omitting the array runs it after every render. Every value used inside the effect should be listed to avoid stale data and bugs.

ПЕРЕВОД

Массив зависимостей сообщает React, когда перезапускать эффект. Пустой массив запускает его один раз после монтирования. Перечисление значений запускает его при изменении любого из них. Отсутствие массива запускает его после каждого рендера. Каждое значение, используемое внутри эффекта, должно быть перечислено, чтобы избежать устаревших данных и багов.

Example / Пример:

```
useEffect(() => { fetchUser(id); }, [id]); // runs when id changes
```

Q67

What is useMemo and when should you use it?

Что такое useMemo и когда его использовать?

ANSWER

useMemo caches the result of an expensive calculation and recomputes it only when its dependencies change, avoiding wasted work on every render. Use it for genuinely costly computations or to keep referential stability of objects/arrays passed to memoized children. Don't overuse it for trivial values.

ПЕРЕВОД

useMemo кэширует результат «дорогого» вычисления и пересчитывает его только при изменении зависимостей, избегая лишней работы на каждом рендере. Используйте для действительно затратных вычислений или для сохранения ссылочной стабильности объектов/массивов, передаваемых в мемоизированные дочерние компоненты. Не злоупотребляйте им для тривиальных значений.

Example / Пример:

```
const sorted = useMemo(() => items.sort(cmp), [items]);
```

Q68

What is useCallback and how is it different from useMemo?

Что такое useCallback и чем он отличается от useMemo?

ANSWER

useCallback memoizes a function so its reference stays stable between renders unless dependencies change. It's mainly used to avoid breaking React.memo on child components or to keep stable effect dependencies. useCallback(fn, deps) is equivalent to useMemo(() => fn, deps): one caches a function, the other a value.

ПЕРЕВОД

useCallback мемоизирует функцию, чтобы её ссылка оставалась стабильной между рендерами, пока не изменятся зависимости. В основном используется, чтобы не «ломать» React.memo у дочерних компонентов или сохранять стабильные зависимости эффектов. useCallback(fn, deps) эквивалентен useMemo(() => fn, deps): один кэширует функцию, другой — значение.

Example / Пример:

```
const onClick = useCallback(() => save(id), [id]);
```

Q69

What is useRef used for?

Для чего используется useRef?

ANSWER

useRef returns a mutable object whose .current persists across renders without triggering a re-render when changed. Two common uses: accessing DOM nodes, and storing mutable values like timer ids, previous values, or instance flags that shouldn't be state.

ПЕРЕВОД

useRef возвращает изменяемый объект, чьё .current сохраняется между рендерами и не вызывает повторный рендер при изменении. Два частых применения: доступ к DOM-узлам и хранение изменяемых значений вроде id таймеров, предыдущих значений или флагов, которые не должны быть состоянием.

Example / Пример:

```
const inputRef = useRef(null);
inputRef.current.focus();
```

Q70

What does useContext do?

Что делает useContext?

ANSWER

useContext subscribes a component to a context and returns its current value, re-rendering when that value changes. It replaces prop drilling for shared data. Note that every consumer re-renders when the context value changes, so split contexts or memoize the value for performance.

ПЕРЕВОД

useContext подписывает компонент на контекст и возвращает его текущее значение, перерисовываясь при изменении этого значения. Он заменяет «прокидывание» props для общих данных. Учтите, что все потребители перерисовываются при изменении значения контекста, поэтому ради производительности разделяйте контексты или мемоизируйте значение.

Example / Пример:

```
const user = useContext(UserContext);
```

Q71

What is useReducer and when is it preferable to useState?

Что такое useReducer и когда он предпочтительнее useState?

ANSWER

useReducer manages state with a reducer function that maps the current state and an action to the next state. It's preferable to useState when state logic is complex, involves multiple sub-values, or the next state depends on the previous one. It also makes state transitions explicit and testable.

ПЕРЕВОД

useReducer управляет состоянием через функцию-редьюсер, которая сопоставляет текущее состояние и действие со следующим состоянием. Он предпочтительнее useState, когда логика состояния сложна, включает несколько подзначений или следующее состояние зависит от предыдущего. Также он делает переходы состояния явными и тестируемыми.

Example / Пример:

```
const [state, dispatch] = useReducer(reducer, initial);
dispatch({ type: 'increment' });
```

Q72

What are custom hooks?

Что такое кастомные (пользовательские) хуки?

ANSWER

A custom hook is a JavaScript function whose name starts with 'use' and that calls other hooks to extract and reuse stateful logic between components. They don't share state, only logic; each call gets its own isolated state. They're the modern way to share behavior, replacing HOCs and render props.

ПЕРЕВОД

Кастомный хук — это JavaScript-функция, имя которой начинается с 'use' и которая вызывает другие хуки, чтобы выделить и переиспользовать логику с состоянием между компонентами. Они разделяют не состояние, а только логику; каждый вызов получает своё изолированное состояние. Это современный способ переиспользовать поведение, заменяющий HOC и render props.

Example / Пример:

```
function useToggle(init = false) {
  const [on, setOn] = useState(init);
  return [on, () => setOn(o => !o)];
}
```

Q73

What is the difference between useEffect and useLayoutEffect?

В чём разница между useEffect и useLayoutEffect?

ANSWER

useEffect runs asynchronously after the browser paints, which is right for most side effects. useLayoutEffect runs synchronously after DOM mutations but before paint, so use it when you must measure or mutate the DOM to avoid a visual flicker. useLayoutEffect can block painting, so use it sparingly.

ПЕРЕВОД

useEffect выполняется асинхронно после отрисовки браузером — это подходит для большинства побочных эффектов. useLayoutEffect выполняется синхронно после изменений DOM, но до отрисовки, поэтому используйте его, когда нужно измерить или изменить DOM, чтобы избежать визуального мерцания. useLayoutEffect может блокировать отрисовку, поэтому применяйте его умеренно.

Example / Пример:

```
useLayoutEffect(() => {
  setWidth(ref.current.offsetWidth);
}, []);
```

Q74

What is the stale closure problem in hooks?

В чём проблема устаревшего замыкания (stale closure) в хуках?

ANSWER

A stale closure happens when an effect or callback captures a value from an old render and keeps using it even after state has updated, because the function was created with that render's scope. Fixes include adding the value to the dependency array or using the functional updater form of setState.

ПЕРЕВОД

Устаревшее замыкание возникает, когда эффект или колбэк захватывает значение из старого рендера и продолжает его использовать даже после обновления состояния, потому что функция была создана в области видимости того рендера. Решения: добавить значение в массив зависимостей или использовать функциональную форму обновления setState.

Example / Пример:

```
// Bug: count is stale
useEffect(() => {
  const id = setInterval(() => setCount(count + 1), 1000);
  return () => clearInterval(id);
}, []);
// Fix: setCount(c => c + 1)
```

Q75

How do hooks replace class lifecycle methods?

Как хуки заменяют методы жизненного цикла классов?

ANSWER

useEffect with an empty dependency array behaves like componentDidMount; its cleanup function behaves like componentWillUnmount; and an effect with dependencies behaves like componentDidUpdate for those values. Instead of thinking in lifecycle phases, you think in terms of synchronizing with specific values.

ПЕРЕВОД

useEffect с пустым массивом зависимостей ведёт себя как componentDidMount; его функция очистки — как componentWillUnmount; а эффект с зависимостями — как componentDidUpdate для этих значений. Вместо мышления в терминах фаз жизненного цикла вы мыслите в терминах синхронизации с конкретными значениями.

Example / Пример:

```
useEffect(() => { /* mount */ return () => { /* unmount */ }; }, []);
```

Q76

What is useImperativeHandle?

Что такое useImperativeHandle?

ANSWER

useImperativeHandle, used with forwardRef, customizes the value exposed to a parent's ref, letting you expose a limited imperative API like focus() or scrollTo() instead of the whole DOM node. It should be used rarely, since declarative props are usually preferable.

ПЕРЕВОД

useImperativeHandle, используемый вместе с forwardRef, настраивает значение, доступное через ref родителя, позволяя предоставить ограниченный императивный API вроде focus() или scrollTo() вместо всего DOM-узла. Его стоит применять редко, поскольку декларативные props обычно предпочтительнее.

Example / Пример:

```
useImperativeHandle(ref, () => ({
  focus: () => inputRef.current.focus(),
}));
```

Q77

What are useTransition and useDeferredValue?

Что такое useTransition и useDeferredValue?

ANSWER

Both are concurrent features for keeping the UI responsive. useTransition marks a state update as non-urgent so React can interrupt it and keep the interface responsive, exposing an isPending flag. useDeferredValue returns a deferred copy of a value that lags behind during heavy updates, useful for expensive filtered lists.

ПЕРЕВОД

Оба — конкурентные возможности для сохранения отзывчивости интерфейса. useTransition помечает обновление состояния как несрочное, чтобы React мог прервать его и держать интерфейс отзывчивым, предоставляя флаг isPending. useDeferredValue возвращает отложенную копию значения, которая отстаёт при тяжёлых обновлениях — полезно для дорогих отфильтрованных списков.

Example / Пример:

```
const [isPending, startTransition] = useTransition();
startTransition(() => setQuery(input));
```

Q78

What is the useId hook for?

Для чего нужен хук useId?

ANSWER

useId generates a unique, stable id that is consistent between server and client rendering, which avoids hydration mismatches in SSR. It's mainly used to link form elements with labels or for accessibility attributes. It is not meant for keys in lists.

ПЕРЕВОД

useId генерирует уникальный стабильный id, согласованный между серверным и клиентским рендерингом, что предотвращает рассогласования гидратации при SSR. В основном используется для связывания элементов формы с подписями или для атрибутов доступности. Он не предназначен для ключей в списках.

Example / Пример:

```
const id = useId();
<label htmlFor={id}>Email</label>
<input id={id} />
```

Redux Toolkit

Вопросы 79–90

Q79

What is Redux and when should you use it?

Что такое Redux и когда его использовать?

ANSWER

Redux is a predictable state container based on a single immutable store, actions that describe changes, and pure reducers that compute the next state. Use it when you have complex global state shared across many components, frequent updates, or need predictable, debuggable state with time-travel tools.

ПЕРЕВОД

Redux — это предсказуемый контейнер состояния, основанный на едином неизменяемом хранилище, действиях (actions), описывающих изменения, и чистых редьюсерах, вычисляющих следующее состояние. Используйте его, когда есть сложное глобальное состояние, разделяемое многими компонентами, частые обновления или нужна предсказуемость и отладка с инструментами «путешествия во времени».

Q80

What is Redux Toolkit and why was it created?

Что такое Redux Toolkit и зачем он создан?

ANSWER

Redux Toolkit (RTK) is the official, recommended way to write Redux. It reduces boilerplate by providing configureStore, createSlice, and createAsyncThunk, includes Immer for 'mutating' immutable updates, sets up good defaults like the Redux DevTools and thunk middleware, and bundles RTK Query for data fetching.

ПЕРЕВОД

Redux Toolkit (RTK) — это официальный рекомендуемый способ писать Redux. Он сокращает шаблонный код, предоставляя configureStore, createSlice и createAsyncThunk, включает Immer для «мутирующих» неизменяемых обновлений, настраивает хорошие умолчания вроде Redux DevTools и thunk-мидлвара и поставляет RTK Query для загрузки данных.

Q81

What is createSlice?

Что такое createSlice?

ANSWER

createSlice generates a slice of state along with its reducer and action creators from a single configuration object containing a name, initial state, and reducers. It uses Immer so you can write 'mutating' logic that produces immutable updates, and it auto-generates action types and creators.

ПЕРЕВОД

createSlice создаёт срез состояния вместе с его редьюсером и создателями действий из одного объекта конфигурации, содержащего имя, начальное состояние и редьюсеры. Он использует Immer, поэтому можно писать «мутирующую» логику, дающую неизменяемые обновления, и автоматически генерирует типы действий и их создателей.

Example / Пример:

```
const slice = createSlice({
  name: 'counter',
  initialState: { value: 0 },
  reducers: { inc: (s) => { s.value++; } },
});
export const { inc } = slice.actions;
```

Q82

What does configureStore do?

Что делает configureStore?

ANSWER

configureStore creates the Redux store with sensible defaults: it combines reducers, automatically adds thunk middleware, enables the Redux DevTools, and includes dev checks for accidental mutations and non-serializable values. It replaces the manual createStore plus applyMiddleware setup.

ПЕРЕВОД

configureStore создаёт хранилище Redux с разумными умолчаниями: объединяет редьюсеры, автоматически добавляет thunk-мидлвар, включает Redux DevTools и проверки в режиме разработки на случайные мутации и несериализуемые значения. Он заменяет ручную настройку createStore и applyMiddleware.

Example / Пример:

```
const store = configureStore({
  reducer: { counter: counterReducer },
});
```

Q83

How does immutability work in Redux Toolkit with Immer?

Как работает неизменяемость в Redux Toolkit с Immer?

ANSWER

RTK uses Immer under the hood, so inside reducers you can write code that looks like direct mutation. Immer tracks those changes on a draft and produces a new immutable state behind the scenes. The rule is: either mutate the draft, or return a brand new value, but never both in one reducer.

ПЕРЕВОД

RTK использует Immer под капотом, поэтому внутри редьюсеров можно писать код, похожий на прямую мутацию. Immer отслеживает эти изменения на черновике (draft) и за кулисами создаёт новое неизменяемое состояние. Правило: либо изменяйте черновик, либо возвращайте совершенно новое значение, но никогда оба варианта в одном редьюсере.

Example / Пример:

```
reducers: {
  addTodo: (state, action) => {
    state.list.push(action.payload); // looks mutable, is safe
  },
}
```

Q84

What is createAsyncThunk?

Что такое createAsyncThunk?

ANSWER

createAsyncThunk handles asynchronous logic like API calls. It accepts an action type prefix and an async function, and automatically dispatches pending, fulfilled, and rejected actions based on the promise outcome. You handle these in extraReducers to update loading, data, and error state.

ПЕРЕВОД

createAsyncThunk обрабатывает асинхронную логику вроде запросов к API. Он принимает префикс типа действия и асинхронную функцию и автоматически диспатчит действия pending, fulfilled и rejected в зависимости от результата промиса. Их обрабатывают в extraReducers для обновления состояния загрузки, данных и ошибки.

Example / Пример:

```
const fetchUser = createAsyncThunk('user/fetch',
  async (id) => (await api.get(id)).data
);
```

Q85

What are extraReducers?

Что такое extraReducers?

ANSWER

extraReducers let a slice respond to actions defined outside of it, most commonly the pending/fulfilled/rejected actions from createAsyncThunk or actions from other slices. You typically use the builder callback syntax with addCase to handle each action type.

ПЕРЕВОД

extraReducers позволяют срезу реагировать на действия, определённые вне него, чаще всего на действия pending/fulfilled/rejected из createAsyncThunk или действия других срезов. Обычно используют синтаксис builder-колбэка с addCase для обработки каждого типа действия.

Example / Пример:

```
extraReducers: (builder) => {
  builder
    .addCase(fetchUser.pending, (s) => { s.loading = true; })
    .addCase(fetchUser.fulfilled, (s, a) => { s.data = a.payload; s.loading = false; });
}
```

Q86

What are useSelector and useDispatch?

Что такое useSelector и useDispatch?

ANSWER

useSelector reads a value from the store and subscribes the component to re-render when that selected slice changes, using strict equality by default. useDispatch returns the store's dispatch function to send actions. Keep selectors narrow and memoize derived data with reselect/createSelector to avoid extra re-renders.

ПЕРЕВОД

useSelector читает значение из хранилища и подписывает компонент на повторный рендер при изменении выбранного среза, по умолчанию используя строгое сравнение. useDispatch возвращает функцию dispatch хранилища для отправки действий. Делайте селекторы узкими и мемоизируйте производные данные через reselect/createSelector, чтобы избежать лишних рендеров.

Example / Пример:

```
const value = useSelector(s => s.counter.value);
const dispatch = useDispatch();
dispatch(inc());
```

Q87

What is RTK Query?

Что такое RTK Query?

ANSWER

RTK Query is a data-fetching and caching layer built into Redux Toolkit. You define an API slice with endpoints, and it auto-generates hooks like useGetPostsQuery that handle loading, caching, deduplication, re-fetching, and cache invalidation via tags. It greatly reduces hand-written async thunks and state.

ПЕРЕВОД

RTK Query — это слой загрузки и кэширования данных, встроенный в Redux Toolkit. Вы описываете API-срез с эндпоинтами, и он автоматически генерирует хуки вроде useGetPostsQuery, которые управляют загрузкой, кэшированием, дедупликацией, повторными запросами и инвалидацией кэша через теги. Это сильно сокращает ручные async-санки и состояние.

Example / Пример:

```
const { data, isLoading } = useGetPostsQuery();
```

Q88

What is middleware in Redux?

Что такое middleware (промежуточное ПО) в Redux?

ANSWER

Middleware sits between dispatching an action and the reducer, letting you intercept, log, modify, or delay actions and run side effects. The thunk middleware (included by default in RTK) lets action creators return functions for async logic. Other examples include logging and analytics middleware.

ПЕРЕВОД

Middleware располагается между диспатчем действия и редьюсером, позволяя перехватывать, логировать, изменять или задерживать действия и выполнять побочные эффекты. Thunk-мидлвар (включённый по умолчанию в RTK) позволяет создателям действий возвращать функции для асинхронной логики. Другие примеры — мидлвары логирования и аналитики.

Example / Пример:

```
const logger = store => next => action => {
  console.log(action.type);
  return next(action);
};
```

Q89

When would you choose Redux over the Context API?

Когда выбрать Redux вместо Context API?

ANSWER

Context is best for low-frequency, relatively static global data like theme or auth, and it can cause many re-renders if the value changes often. Redux suits complex, frequently-updated state with many actions, where you benefit from middleware, DevTools, selectors, and structured update logic. They can also be combined.

ПЕРЕВОД

Context лучше всего подходит для редко меняющихся, относительно статичных глобальных данных вроде темы или авторизации, и при частом изменении значения может вызывать много повторных рендеров. Redux подходит для сложного, часто обновляемого состояния со множеством действий, где полезны middleware, DevTools, селекторы и структурированная логика обновлений. Их также можно сочетать.

Q90

What is createEntityAdapter and normalization?

Что такое createEntityAdapter и нормализация?

ANSWER

Normalization stores collections as a lookup of items by id plus an array of ids, instead of nested arrays, which makes updates and lookups efficient. createEntityAdapter provides prebuilt reducers and selectors (addOne, updateOne, selectAll) to manage this normalized shape consistently.

ПЕРЕВОД

Нормализация хранит коллекции как словарь элементов по id плюс массив id, вместо вложенных массивов, что делает обновления и поиск эффективными. createEntityAdapter предоставляет готовые редьюсеры и селекторы (addOne, updateOne, selectAll) для согласованного управления этой нормализованной структурой.

Example / Пример:

```
const adapter = createEntityAdapter();
adapter.addOne(state, item);
const { selectAll } = adapter.getSelectors();
```

Next.js

Вопросы 91–100

Q91

What is Next.js and what are its benefits?

Что такое Next.js и каковы его преимущества?

ANSWER

Next.js is a React framework for production that adds server-side rendering, static generation, file-based routing, API routes, image and font optimization, and built-in bundling. The benefits are better SEO and performance, flexible rendering strategies per page, and a strong developer experience out of the box.

ПЕРЕВОД

Next.js — это React-фреймворк для продакшена, добавляющий серверный рендеринг, статическую генерацию, файловую маршрутизацию, API-роуты, оптимизацию изображений и шрифтов и встроенную сборку. Преимущества — лучшее SEO и производительность, гибкие стратегии рендеринга для каждой страницы и хороший опыт разработки «из коробки».

Q92

What is the difference between SSR, SSG, ISR, and CSR?

В чём разница между SSR, SSG, ISR и CSR?

ANSWER

CSR renders in the browser after JS loads. SSR renders HTML on the server per request, good for fresh, personalized data. SSG pre-renders pages at build time for the fastest delivery of static content. ISR extends SSG by regenerating static pages in the background after a set interval, combining speed with freshness.

ПЕРЕВОД

CSR рендерит в браузере после загрузки JS. SSR рендерит HTML на сервере при каждом запросе — подходит для свежих, персонализированных данных. SSG предварительно генерирует страницы во время сборки для максимально быстрой выдачи статического контента. ISR расширяет SSG, перегенерируя статические страницы в фоне через заданный интервал, сочетая скорость и актуальность.

Q93

What is the difference between getServerSideProps and getStaticProps?

В чём разница между getServerSideProps и getStaticProps?

ANSWER

These are data-fetching functions in the Pages Router. getStaticProps runs at build time to pre-render a page as static (SSG), optionally with revalidation for ISR. getServerSideProps runs on every request on the server (SSR), giving up-to-date data at the cost of slower responses. Both run only on the server.

ПЕРЕВОД

Это функции загрузки данных в Pages Router. getStaticProps выполняется во время сборки для предварительного рендеринга страницы как статической (SSG), при необходимости с revalidate для ISR. getServerSideProps выполняется при каждом запросе на сервере (SSR), давая актуальные данные ценой более медленного ответа. Обе работают только на сервере.

Example / Пример:

```
export async function getStaticProps() {
  return { props: { data }, revalidate: 60 };
}
```

Q94

What is the difference between the App Router and the Pages Router?

В чём разница между App Router и Pages Router?

ANSWER

The Pages Router (pages/) is the original system using getStaticProps/getServerSideProps. The App Router (app/), default since Next 13, uses React Server Components, nested layouts, loading and error files, and data fetching directly in async components with fetch. The App Router is the recommended approach for new apps.

ПЕРЕВОД

Pages Router (pages/) — это первоначальная система, использующая getStaticProps/getServerSideProps. App Router (app/), стандартный с Next 13, использует React Server Components, вложенные layout-ы, файлы loading и error и загрузку данных прямо в async-компонентах через fetch. App Router рекомендуется для новых приложений.

Q95

What is the difference between Server Components and Client Components?

В чём разница между Server Components и Client Components?

ANSWER

In the App Router, components are Server Components by default: they render on the server, can fetch data directly, ship no JS to the client, and reduce bundle size, but can't use state or browser APIs. Adding 'use client' makes a Client Component that runs in the browser and supports hooks, state, and event handlers.

ПЕРЕВОД

В App Router компоненты по умолчанию являются Server Components: они рендерятся на сервере, могут напрямую загружать данные, не отправляют JS на клиент и уменьшают размер бандла, но не могут использовать состояние или браузерные API. Директива 'use client' делает Client Component, который выполняется в браузере и поддерживает хуки, состояние и обработчики событий.

Example / Пример:

```
'use client';
import { useState } from 'react';
// now state and hooks are allowed
```

Q96

How does routing work in Next.js, including dynamic routes?

Как работает маршрутизация в Next.js, включая динамические маршруты?

ANSWER

Routing is file-system based: folders and files map to URLs. In the App Router a folder with a page.tsx becomes a route. Dynamic segments use square brackets like [id], catch-all uses [...slug], and you navigate with the Link component or the useRouter hook for programmatic navigation.

ПЕРЕВОД

Маршрутизация основана на файловой системе: папки и файлы сопоставляются с URL. В App Router папка с page.tsx становится маршрутом. Динамические сегменты используют квадратные скобки вроде [id], «ловящие всё» — [...slug], а навигация выполняется компонентом Link или хуком useRouter для программной навигации.

Example / Пример:

```
// app/posts/[id]/page.tsx
export default function Post({ params }) {
  return <h1>{params.id}</h1>;
}
```

Q97

What are API routes / route handlers in Next.js?

Что такое API-роуты / обработчики маршрутов в Next.js?

ANSWER

Next.js lets you build backend endpoints inside the same project. In the App Router you export functions named after HTTP methods (GET, POST) from a route.ts file, returning a Response. They run server-side only, so they're safe for secrets, database access, or proxying external APIs.

ПЕРЕВОД

Next.js позволяет создавать бэкенд-эндпоинты внутри того же проекта. В App Router вы экспортируете функции, названные по HTTP-методам (GET, POST), из файла route.ts, возвращающие Response. Они выполняются только на сервере, поэтому безопасны для секретов, доступа к базе данных или проксирования внешних API.

Example / Пример:

```
// app/api/users/route.ts
export async function GET() {
  return Response.json({ users: [] });
}
```

Q98

How does image optimization work with next/image?

Как работает оптимизация изображений с next/image?

ANSWER

The next/image component automatically optimizes images: it serves modern formats like WebP, resizes for the device, lazy-loads by default, and reserves space to prevent layout shift. You specify width and height (or fill) and it handles responsive sizes, improving Core Web Vitals.

ПЕРЕВОД

Компонент next/image автоматически оптимизирует изображения: отдаёт современные форматы вроде WebP, изменяет размер под устройство, лениво загружает по умолчанию и резервирует место для предотвращения сдвига вёрстки. Вы задаёте width и height (или fill), а он управляет адаптивными размерами, улучшая Core Web Vitals.

Example / Пример:

```
import Image from 'next/image';
<Image src="/a.jpg" width={400} height={300} alt="" />
```

Q99

How does data fetching and caching work in the App Router?

Как работает загрузка данных и кэширование в App Router?

ANSWER

In the App Router you fetch data directly in async Server Components using the extended fetch API. Next.js caches fetch results by default; you control behavior with options like cache: 'no-store' for dynamic data or next: { revalidate: N } for time-based revalidation (ISR-style).

ПЕРЕВОД

В App Router вы загружаете данные напрямую в async Server Components, используя расширенный fetch API. Next.js по умолчанию кэширует результаты fetch; поведением управляют опции вроде cache: 'no-store' для динамических данных или next: { revalidate: N } для ревалидации по времени (в стиле ISR).

Example / Пример:

```
const res = await fetch(url, { next: { revalidate: 60 } });
const data = await res.json();
```

Q100

What is middleware in Next.js and how do you handle SEO/metadata?

Что такое middleware в Next.js и как работать с SEO/метаданными?

ANSWER

Next.js middleware runs before a request is completed, at the edge, ideal for redirects, authentication, geolocation, or rewrites. For SEO, the App Router lets you export a static metadata object or a generateMetadata function from a page or layout to set the title, description, Open Graph tags, and more per route.

ПЕРЕВОД

Middleware в Next.js выполняется до завершения запроса, на edge, что идеально для редиректов, авторизации, геолокации или rewrite-ов. Для SEO App Router позволяет экспортировать статический объект metadata или функцию generateMetadata из страницы или layout, чтобы задавать title, description, Open Graph-теги и прочее для каждого маршрута.

Example / Пример:

```
export const metadata = {
  title: 'Home',
  description: 'Welcome',
};
```

Glossary / Словарь терминов

Ключевые термины фронтенд-разработки

Hoisting	Поднятие объявлений переменных и функций в начало области видимости.
Closure	Замыкание — функция, запоминающая переменные из внешней области видимости.
Scope	Область видимости — где доступны переменные (глобальная, функциональная, блочная).
Event loop	Событийный цикл — механизм обработки асинхронных задач в JS.
Callback	Колбэк — функция, передаваемая в другую функцию для вызова позже.
Promise	Промис — объект, представляющий результат будущей асинхронной операции.
Coercion	Приведение типов — автоматическое преобразование одного типа в другой.
Prototype	Прототип — объект, от которого другой объект наследует свойства и методы.
Immutable	Неизменяемый — объект/значение, которое нельзя изменить после создания.
Mutation	Мутация — изменение объекта или массива на месте.
Pure function	Чистая функция — без побочных эффектов, результат зависит только от аргументов.
Side effect	Побочный эффект — изменение состояния вне функции (запрос, DOM, лог).
Higher-order function	Функция высшего порядка — принимает или возвращает функцию.
Currying	Каррирование — превращение функции многих аргументов в цепочку функций.
Debounce	Дебаунс — откладывание вызова до паузы между событиями.
Throttle	Тротлинг — ограничение частоты вызова функции по времени.
Virtual DOM	Виртуальный DOM — представление UI в памяти для эффективного обновления.
Reconciliation	Согласование — процесс сравнения старого и нового virtual DOM.
Component	Компонент — переиспользуемый независимый блок интерфейса.
Props	Пропсы — входные данные, передаваемые компоненту (только для чтения).
State	Состояние — внутренние изменяемые данные компонента.
Hook	Хук — функция для использования состояния и эффектов в компонентах.
Render	Рендер — процесс вычисления и отображения UI компонента.
Re-render	Повторный рендер — обновление компонента при изменении state/props.
Memoization	Мемоизация — кэширование результата вычисления для повторного использования.
Prop drilling	Прокидывание пропсов — передача данных через много уровней компонентов.

Context	Контекст — способ передачи данных без prop drilling.
Reducer	Редьюсер — чистая функция (state, action) => newState.
Action	Действие — объект, описывающий что произошло, с полем type.
Dispatch	Диспатч — отправка действия в хранилище Redux.
Store	Хранилище — единый объект состояния приложения в Redux.
Selector	Селектор — функция извлечения данных из состояния.
Slice	Срез — часть состояния Redux с редьюсерами и действиями (RTK).
Thunk	Санк — функция для асинхронной логики в Redux.
Middleware	Промежуточное ПО — перехватывает действия между dispatch и reducer.
Normalization	Нормализация — хранение данных по id для эффективного доступа.
SSR	Серверный рендеринг — генерация HTML на сервере при запросе.
SSG	Статическая генерация — предварительный рендер страниц при сборке.
ISR	Инкрементальная статическая регенерация — обновление статики в фоне.
CSR	Клиентский рендеринг — отрисовка в браузере после загрузки JS.
Hydration	Гидратация — «оживление» серверного HTML на клиенте через JS.
Routing	Маршрутизация — сопоставление URL с компонентами/страницами.
Bundle	Бандл — собранный файл(ы) приложения для доставки в браузер.
Code splitting	Разделение кода — разбиение бандла на части, загружаемые по требованию.
Lazy loading	Ленивая загрузка — загрузка ресурса только при необходимости.
Type inference	Вывод типов — автоматическое определение типов компилятором TS.
Generic	Дженерик — обобщённый тип-параметр для переиспользуемого кода.
Type guard	Сужающая проверка типа в TypeScript.
Interface	Интерфейс — описание формы объекта в TypeScript.
Union type	Объединение типов — значение может быть одним из нескольких типов.

Common phrases / Распространённые фразы

Полезные фразы для собеседования на английском

Let me walk you through my approach.	Позвольте описать мой подход.
That's a great question.	Это отличный вопрос.
Could you clarify the requirements?	Не могли бы вы уточнить требования?
Let me think about this for a moment.	Дайте мне немного подумать.
Off the top of my head...	Навскидку / не задумываясь...
It depends on the use case.	Это зависит от конкретной задачи.
In a nutshell...	Если коротко / в двух словах...
Under the hood, it works like this.	Под капотом это работает так.
Let me give you an example.	Позвольте привести пример.
To put it simply...	Проще говоря...
There's a trade-off here.	Здесь есть компромисс.
A common pitfall is...	Распространённая ошибка — это...
Best practice is to...	Хорошая практика — это...
I'd refactor this to...	Я бы отрефакторил это в...
This improves performance.	Это улучшает производительность.
Let's break it down.	Давайте разберём это по частям.
As far as I know...	Насколько мне известно...
Correct me if I'm wrong.	Поправьте меня, если я ошибаюсь.
That makes sense.	Это имеет смысл / логично.
I'm not sure, but I would assume...	Я не уверен, но предположил бы...
Let me double-check that.	Позвольте мне это перепроверить.
In my previous role, I...	На предыдущей должности я...
I'm comfortable with...	Я уверенно владею...
I'd like to highlight that...	Хочу подчеркнуть, что...
To sum up...	Подводя итог...
Does that answer your question?	Это отвечает на ваш вопрос?
I'd approach it step by step.	Я подошёл бы к этому пошагово.
Edge case	Граничный / крайний случай.
Boilerplate code	Шаблонный (повторяющийся) код.
Out of the box	Из коробки / по умолчанию.